

AI Failure Analysis Copilot

An automated AI assistant that triages failed Playwright tests in seconds with root-cause diagnosis & suggested fixes.

Engine: Microsoft Playwright **AI Intelligence:** Local Ollama (Offline & Free) **Test Suite:** 60 Real Tests (7 Web Apps/APIs)

Data Privacy: 100% On-Premise / Zero Cloud Data

TESTS EVALUATED

60

Across 7 real web apps & APIs

FAILURES TRIAGED

44

100% root-cause coverage

TRIAGE SPEED

2.5 min

vs ~11-14 hrs manual labor

RUNNING COST

\$0.00

100% offline & local AI

1. What This Tool Is About (In Short)

When automated tests fail, software engineers usually waste **10 to 15 hours every week** reading cryptic logs and opening screenshots.

👉 **This tool automatically reads the failure logs and screenshots, tells you the exact root cause in 3 seconds (e.g. Real Bug vs. Changed Button vs. Slow Network), and suggests how to fix the code.**

- ⚡ 3-Sec AI Triage
- 🖼️ Screenshot Vision Diagnostics
- 📄 Flaky Stability Score
- 🔗 1-Click Code Fix Diff

2. Some Little Proof (From Real 60-Test Execution)

🐛 Application Bug		17 (39%)	Total Tests Evaluated	60 Tests
🔍 Locator / Selector		15 (34%)	Passing Tests	16 Passed (26.7%)
⌚ Timing / Sync Issue		7 (16%)	Failures Triaged	44 Failures (73.3%)
🌐 Environment / Infra		2 (5%)	Classification Coverage	100% (0 Unclassified)
📜 Test Script Bug		2 (5%)	AI Triage Duration	~2.5 Minutes
📁 Test Data Issue		1 (2%)	Cloud API Bill	\$0.00 (Offline)

3. Tech Stack (To Build Tool & Framework)

LAYER	TECHNOLOGY	PURPOSE
Test Framework	Playwright + TypeScript	Automated browser testing across Chromium with screenshot/log capture
AI / LLM Engine	Local Ollama (qwen2.5-coder)	100% offline root-cause triage & vision analysis (\$0 cloud cost)
Backend Server	Node.js + Express 5 + TypeScript	REST APIs, process management (taskkill), report generator
Live Streaming	Server-Sent Events (SSE)	Real-time failure-by-failure live triage terminal streaming
History & Analytics	SQLite (better-sqlite3)	Flaky test history tracking and stability scoring (0-100%)
Auto-Fix Engine	Diff Library (diff) + Zod	Generates unified git diffs with 1-click .bak safe rollback
Web Dashboard	HTML5 + CSS3 + Vanilla JS	Executive UI at localhost:3000 with interactive triage cards & reports

How to Pull, Setup & Use the Tool

Complete instructions for developers, QA testers, and evaluators to run the tool from scratch.

1 How to Pull the Code (Repository URL)

Clone the official project repository from GitHub using Git:

```
git clone https://github.com/KiranAcad/AI-FEST-ACAD-QA.git
cd AI-FEST-ACAD-QA
```

• **Repository:** <https://github.com/KiranAcad/AI-FEST-ACAD-QA.git> • **Branch:** master

2 Prerequisites & Installation

Install required dependencies for both the copilot backend and Playwright test suite:

```
# 1. Install project dependencies
npm install

# 2. (Optional) Ensure Ollama is running locally with coding model
ollama pull qwen2.5-coder:1.5b
```

• **Requirements:** Node.js >= 18.0.0, Git, and Ollama (for offline local AI inference).

3 How to Start the Project & Dashboard

Launch the web dashboard server with a single npm command:

```
npm run dashboard
```

Once started, open your web browser and navigate to: <http://localhost:3000>

4 How to Use the Tool (Dashboard Features)

The web application at <http://localhost:3000> provides 4 core workflow tabs:



Dashboard

High-level executive metrics, category distribution charts & stability trends.



Failure Triage

Browse triaged failures, view AI explanations, inspect screenshots, and apply code diffs.



Run Tests

1-click "🔥 Run Tests & Analyze" button. Terminal streams tests and live AI triage!



Reports

Preview or download standalone interactive HTML reports (77 KB), JSON data, or Markdown.

5 Quick CLI Cheat Sheet (For CI/CD & Power Users)

COMMAND	WHAT IT DOES
<code>npm run test:real</code>	Executes all 60 Playwright tests across 7 web applications and saves <code>results.json</code>
<code>npm run analyze:ollama</code>	Runs AI batch triage from command line using local Ollama model
<code>npm run test:and:analyze:ollama</code>	Full end-to-end automated pipeline: executes tests, captures failures, and generates reports